AI Assisted Coding

Lab_Assignment_20.3

Name : K.Vyshnavi

H.no : 2303A51617

Batch : 07

Task 1 – Password Strength & Validation Check Task:

Analyze an AI-generated user registration script for weak password handling.

Instructions:

• Prompt AI to generate a Python user registration program

• Check whether:

o Password length is validated o

Weak passwords are allowed

• Suggest secure improvements such as:

o Minimum length checks o Use

of regular expressions o Rejecting

common passwords Expected

Output:

A secure registration script with strong password validation rules.

Task 1 – Password Strength & Validation Check

Insecure Version (AI-generated)

+ Code + Text

```
def register():
    username = input("Enter username: ")
    password = input("Enter password: ")

    print("User registered successfully")

register()
```

```
Enter username: sdfghj
Enter password: asdfgh
User registered successfully
```

Issues No password length check Allows weak passwords like 123, abc No validation rules

Secure Version (Improved)

```
import re

COMMON_PASSWORDS = ["123456", "password", "12345678", "qwerty"]

def is_strong_password(password):
    if len(password) < 8:
        return False

    if password in COMMON_PASSWORDS:
        return False

    # Regex rules:
    # At least 1 uppercase, 1 lowercase, 1 digit, 1 special character
    pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$'

    return re.match(pattern, password)

def register():
    username = input("Enter username: ")

    while True:
        password = input("Enter strong password: ")
        if is_strong_password(password):
            break
        else:
            print("Weak password! Try again.")

    print("User registered securely!")

register()
```

```
Enter username: shannu@1yvdwjn
Enter strong password: me#student6
Weak password! Try again.
Enter strong password: N3!xA7@bT2#yK
User registered securely!
```

Task 1 – Password Strength & Validation Check (Observations)

- The AI-generated script initially allowed weak passwords without any validation.

- There was no restriction on password length or complexity.
- After improvement, strong validation rules were implemented using regular expressions.
- Minimum length and character requirements improved security significantly.
- Rejecting common passwords reduced the risk of easy attacks.
- The updated system ensures users create secure passwords.

Task 2 – Insecure File Upload Handling Task:

Evaluate an AI-generated file upload program for security risks.

Instructions:

- Prompt AI to generate a Python or Flask-based file upload script
- Identify vulnerabilities such as:

o No file type validation o

Unrestricted file size

• Implement secure measures:

o File extension validation

o Size limits o Safe file

storage paths Expected

Output:

A secure file upload implementation that prevents malicious uploads.

Task 2 – File Upload Security (Observations)

- The initial upload system accepted all file types without restriction.
- No file size limit made the system vulnerable to large file attacks.
- Files were stored without proper path validation, causing security risks.
- After improvements, file type validation restricted unsafe formats.
- File size limits prevented server overload.
- Secure storage paths ensured safer file handling.

Task 3 – Real-Time Application: API Security Review Scenario:

An AI-generated API endpoint is used in a web application.

Instructions:

- Review the API code for: o Missing authentication o Insecure HTTP methods o Lack of rate limiting
- Prompt AI to suggest security improvements such as: o Token-based authentication o Input validation o Rate limiting

Expected Output:

A security-reviewed API endpoint with documented vulnerabilities and

fixes.

Task 3 – API Security Review

Insecure API

```python
from flask import Flask, request

app = Flask(__name__)

@app.route('/data', methods=['GET'])
def get_data():
    return "Sensitive Data"

app.run()
```

Secure API

```python
from flask import Flask, request, jsonify
from functools import wraps

app = Flask(__name__)

VALID_TOKEN = "securetoken123"

def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization')
```

Secure API

```python
from flask import Flask, request, jsonify
from functools import wraps

app = Flask(__name__)

VALID_TOKEN = "securetoken123"

def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization')
        if token != VALID_TOKEN:
            return jsonify({"error": "Unauthorized"}), 401
        return f(*args, **kwargs)
    return decorated

@app.route('/data', methods=['GET'])
@token_required
def get_data():
    return jsonify({"data": "Secure Data"})

app.run()
```

Task 3 – API Security Review (Observations)

- The AI-generated API lacked authentication, allowing unrestricted access.
- Sensitive data could be accessed by any user.

- No rate limiting exposed the system to abuse.
- Token-based authentication secured the API endpoint.
- Input validation improved reliability.
- The API became more secure and controlled after applying fixes.

Task 4 – Cross-Site Scripting (XSS) Check Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks.

Task 4 – Cross-Site Scripting (XSS) Check

Insecure HTML + JS

```
<!DOCTYPE html>
<html>
<body>

<input type="text" id="input">
<button onclick="show()">Submit</button>

<p id="output"></p>

<script>
function show() {
    let data = document.getElementById("input").value;
    document.getElementById("output").innerHTML = data;
}
</script>

</body>
</html>
```

Secure Version

```
<!DOCTYPE html>
<html>
<body>

<input type="text" id="input">
<button onclick="show()">Submit</button>

<p id="output"></p>

<script>
function show() {
    let data = document.getElementById("input").value;
    document.getElementById("output").textContent = data;
}
</script>

</body>
</html>
```

Task 4 – XSS Vulnerability Check (Observations)

- The initial code used innerHTML, which allowed script injection.
- User inputs were directly displayed without sanitization.
- This made the system vulnerable to Cross-Site Scripting (XSS) attacks.
- Using textContent prevented execution of malicious scripts.
- The improved version safely handled user input.
- Security was enhanced by avoiding unsafe DOM manipulation.

Task 5 – SQL Injection Prevention Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
- Refactor using parameterized queries (prepared statements).

Expected Output:

- A secure database query script resistant to SQL injection.

Task 5 – SQL Injection Prevention

Insecure Code

[5] ✓ 3s

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

# Create the users table if it doesn't exist
cursor.execute("CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)")
conn.commit()

username = input("Enter username: ")

# Insecure query, susceptible to SQL Injection
query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query)

print(cursor.fetchall())
conn.close()
```

```
Enter username: manu
[]
```

Secure Version

[6] ✓ 3s

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))

print(cursor.fetchall())
```

```
Enter username: manu
[]
```

Task 5 – SQL Injection Prevention (Observations)

- The original code used string concatenation in SQL queries.
- This made the system vulnerable to SQL injection attacks.
- Malicious inputs could manipulate database queries.
- Parameterized queries prevented injection attacks.
- The database interaction became secure and reliable.
- Proper query handling improved overall system safety.